

Multithreaded Web Server with Database Integration

Complete Project Documentation

This guide explains the **whole project from start to finish**: what each part does, **why** it is built that way, and what would go wrong without it. It assumes basic C++ and SQL, and explains everything else along the way.

Everything in one place: how the server works and why, the database design with ER diagrams, limitations, and viva preparation.

Team: Shoyam Bishnoi (Leader) · Sneha Negi · Ashutosh · Aradhana K. Jaiswal

Table of contents

1. [The project in one minute](#)
 2. [A real-world analogy](#)
 3. [Technology choices and why](#)
 4. [Files in the repository](#)
 5. [How to run it](#)
 6. [Big-picture architecture](#)
 7. [Server startup](#)
 8. [Multithreading: thread pool, queue, semaphore, mutex](#)
 9. [Life of one request](#)
 10. [Understanding HTTP: what the server actually reads](#)
 11. [The router](#)
 12. [Building pages \(HTML\)](#)
 13. [Users: signup and login](#)
 14. [Sessions: how the server remembers you](#)
 15. [Products page](#)
 16. [Cart](#)
 17. [Checkout: the transaction](#)
 18. [Orders page](#)
 19. [Database design and ER diagrams](#)
 20. [OS and DBMS concepts used \(syllabus map\)](#)
 21. [Known limitations and how to improve them](#)
 22. [Viva / review questions](#)
 23. [Glossary](#)
-

1. The project in one minute

This is a **small online shop** (like a mini Amazon). The interesting part is not the shop itself, but **how it is built**:

- We wrote our **own web server in C++**, directly on top of network sockets. There is no Apache, Nginx, Node.js or framework.
- The server can serve **many users at the same time** using a **thread pool** (4 worker threads).
- All data (users, products, carts, orders, payments, login sessions) is stored in **MySQL**.
- Placing an order is done as a **database transaction**. Either *everything* happens (order created, stock reduced, payment recorded, cart emptied) or *nothing* happens. Two people buying the last item at the same time can never both get it.

The goal was to see **Operating Systems** concepts (threads, synchronization, producer–consumer, critical sections) and **DBMS** concepts (transactions, ACID, locking, normalization) working together in one real program.

2. A real-world analogy

Think of the server as a **restaurant**:

Restaurant	Our server	Code
Customer walks in	Browser opens a TCP connection	<code>accept()</code>
Receptionist at the door	Main thread : only accepts customers and seats them in the queue	<code>main()</code> loop
Waiting list / order slips	Request queue	<code>requestQueue</code>
Bell that rings when a slip is added	Semaphore : wakes a sleeping waiter	<code>requestSemaphore</code>
Rule: only one hand on the slip pile at a time	Mutex on the queue	<code>queueMutex</code>
4 waiters	4 worker threads	<code>worker()</code>
One shared kitchen with one chef	One MySQL connection , used by one waiter at a time	<code>dbConn</code> + <code>dbMutex</code>
Loyalty card the customer shows each visit	Session cookie	<code>session_id</code>
Bill must be fully paid or fully cancelled	Transaction (COMMIT / ROLLBACK)	<code>checkoutCart()</code>

The receptionist never cooks. That is why the restaurant keeps accepting new customers even when every waiter is busy.

3. Technology choices and why

Technology	Used for	Why we chose it
C++	Entire server	Gives direct control over threads, memory and sockets. Nothing is hidden by a framework, so the OS concepts are visible in the code.
Winsock2 (<code>winsock2.h</code>)	Networking (TCP sockets)	The Windows socket API. We read raw bytes and parse HTTP ourselves, which shows how a web server really works underneath.
<code>std::thread</code>	Worker threads	Standard C++ threads, portable and simple.
<code>std::mutex</code> + <code>lock_guard</code>	Mutual exclusion	<code>lock_guard</code> unlocks automatically when the function returns, even on an early <code>return</code> . It is impossible to "forget to unlock".
Windows semaphore (<code>CreateSemaphore</code>)	Counting queued requests	A counting semaphore is the textbook tool for producer-consumer. Idle threads sleep (0% CPU) instead of looping and checking.
<code>thread_local</code>	Per-request data (session, user name)	Each thread gets its own copy, so concurrent requests never mix up users.
MySQL + C API (<code>mysql.h</code> , <code>libmysql.dll</code>)	Persistent storage	A relational DB with real transactions (InnoDB), row locks and foreign keys, which is exactly what the DBMS part needs.
Server-generated HTML	User interface	The server builds each page as a string. There is no separate frontend build step, so the whole system is one program.
Cookies	Remembering logged-in users	HTTP itself is stateless. Cookies are the standard way to carry a session ID automatically.
g++ (MinGW)	Compiler	Free, and works with the VS Code task in <code>.vscode/tasks.json</code> .

4. Files in the repository

```
multithreaded-webserver-db/
├─ main.cpp           ← the entire server (~2,500 lines)
├─ ecommerce_db.sql   ← creates the database, 9 tables, and sample data
├─ server.exe         ← compiled server (Windows)
├─ libmysql.dll       ← MySQL client library; must sit next to server.exe
├─ .vscode/
│   ├─ tasks.json     ← "build" task: g++ command with MySQL include/lib paths
│   └─ launch.json    ← debugger configuration
└─ docs/
    ├─ README.md      ← project overview
    ├─ PROJECT_DOCUMENTATION.md ← this file (everything in one)
    ├─ PROJECT_DOCUMENTATION.pdf ← same, as a single PDF
    ├─ er_diagram*.svg ← the diagrams
    ├─ gen_er_diagram.py ← regenerates the ER diagrams
    ├─ diagrams/      ← flow diagrams used in this guide (.svg + Mermaid .mmd source)
    └─ architecture*.png ← architecture images
```

`main.cpp` is organised into clearly labelled sections, top to bottom:

Section	Main functions	Purpose
CONFIG	<code>DB_HOST</code> , <code>DB_USER</code> , ...	Database connection settings
THREAD POOL	<code>Request</code> , <code>requestQueue</code> , <code>queueMutex</code> , <code>requestSemaphore</code>	Shared work queue
DB	<code>initDatabase()</code> , <code>esc()</code>	Connect to MySQL; escape user input
REQUEST CONTEXT	<code>tlSessionId</code> , <code>tlUserName</code> , <code>tlExtraHeaders</code> , <code>isLoggedIn()</code>	Per-thread "who is this request from"
HTML HELPERS	<code>htmlEscape()</code> , <code>pageWrapper()</code>	Safe text output; common page layout
SESSION MANAGEMENT	<code>generateSessionId()</code> , <code>createSession()</code> , <code>getSessionUserId()</code> , <code>logoutSession()</code> , <code>getCookie()</code> , <code>getUserName()</code>	Login sessions
SESSION PAGE	<code>buildSessionPage()</code>	"My Session" page
PRODUCTS / SIGNUP / LOGIN	<code>buildProductsPage()</code> , <code>handleSignup()</code> , <code>handleLogin()</code>	Catalogue and accounts
CART	<code>getActiveCartIdLocked()</code> , <code>buildCartPage()</code> , <code>addCartItem()</code> , <code>removeCartItem()</code>	Shopping cart
ORDERS AND PAYMENTS	<code>checkoutCart()</code> , <code>buildOrdersPage()</code>	Checkout transaction and history
HTTP PARSING	<code>parseMethod()</code> , <code>parsePath()</code> , <code>parseQuery()</code> , <code>parseFormBody()</code> , <code>urlDecode()</code> , <code>contentLength()</code>	Understanding the raw request

Section	Main functions	Purpose
HTTP RESPONSE	<code>makeHttpResponse()</code>	Building the reply
ROUTER	<code>route()</code>	Deciding which function handles a URL
WORKER	<code>worker()</code>	What each thread does forever
MAIN	<code>main()</code>	Startup and the accept loop

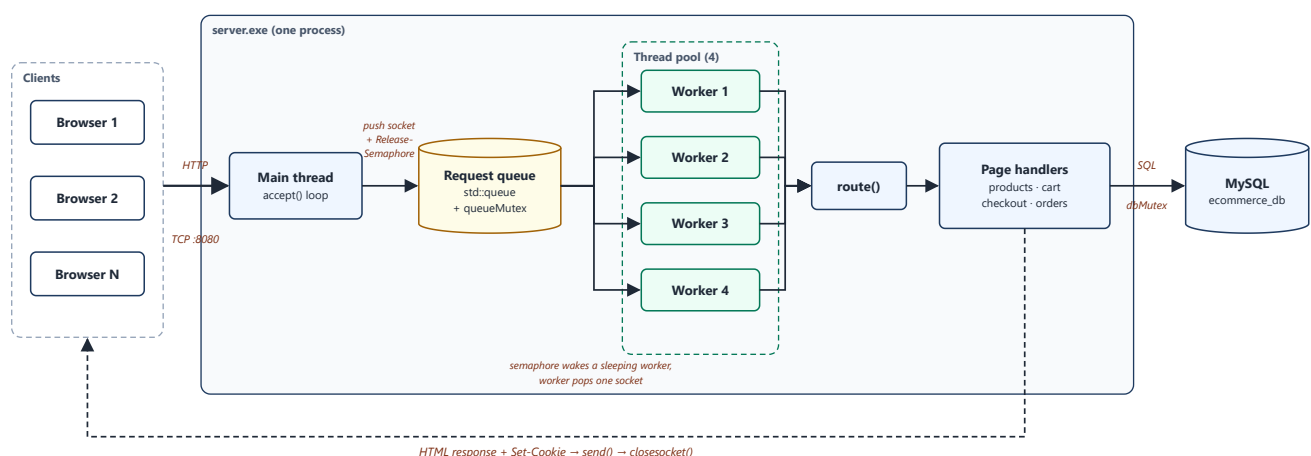
5. How to run it

1. **Install MySQL Server 8.0** (the build task expects `C:/Program Files/MySQL/MySQL Server 8.0/`).
2. **Create the database:** `powershell mysql -u root -p < ecommerce_db.sql`
3. **Set your DB password** in `main.cpp` (`DB_PASS`, near the top).
4. **Build** in VS Code with `Ctrl+Shift+B`, or run: `powershell g++ -g main.cpp -o server.exe -I "C:/Program Files/MySQL/MySQL Server 8.0/include" -L "C:/Program Files/MySQL/MySQL Server 8.0/lib" -lmysql -lws2_32 -static-libgcc -static-libstdc++`
5. **Run** `server.exe`. `libmysql.dll` must be in the same folder. You should see: `[DB] Connected to MySQL successfully. Server on http://localhost:8080`
6. Open **`http://localhost:8080`** in a browser. Each request is logged as `[W2] GET /products`, where `W2` is the worker thread that handled it.

Why `-lws2_32`? It links the Windows socket library. **Why `-lmysql`?** It links the MySQL client library. **Why `-static-libgcc -static-libstdc++`?** So `server.exe` runs on PCs without MinGW installed.

6. Big-picture architecture

Architecture: request path through the server



The flow in words: **Browser** → **main thread accepts** → **queue** → **free worker picks it up** → **reads and parses HTTP** → **router** → **handler** → **MySQL** → **HTML response** → **browser**.

7. Server startup (`main()`)

When you run `server.exe`, `main()` does this **once**:

Step	Code	What it does	Why
1	<code>WSAStartup(MAKEWORD(2,2), ...)</code>	Initialises Windows networking	On Windows, you must do this before using any socket function.
2	<code>initDatabase()</code>	<code>mysql_init</code> + <code>mysql_real_connect</code> + <code>utf8mb4</code>	Connect once at startup. Connecting per request would be slow. <code>utf8mb4</code> supports all characters, including ₹ and emoji.
3	<code>CreateSemaphore(NULL, 0, 1000, NULL)</code>	Counting semaphore, starting at 0	0 means "no requests waiting yet". Workers that wait on it go to sleep.
4	<code>socket(AF_INET, SOCK_STREAM, IPPROTO_TCP)</code>	Creates a TCP socket	HTTP runs over TCP, which is reliable and keeps bytes in order.
5	<code>bind(... port 8080, INADDR_ANY)</code>	Attaches the socket to port 8080 on all network interfaces	So browsers can reach <code>localhost:8080</code> .
6	<code>listen(serverSocket, SOMAXCONN)</code>	Starts accepting connections	<code>SOMAXCONN</code> lets the OS queue as many pending connections as it allows.
7	Start 4 <code>worker</code> threads	The thread pool	Threads are created once and reused (see §8).
8	<code>while(true) { accept → push → ReleaseSemaphore }</code>	Main loop, runs forever	The main thread only <i>accepts</i> connections, so it is always ready for the next customer.

If any step fails, the program cleans up what it already created (closes the socket, semaphore and DB connection, then calls `WSACleanup`) and exits. This avoids leaking resources.

8. Multithreading: thread pool, queue, semaphore, mutex

This is the **Operating Systems heart** of the project.

8.1 Why multithreading at all?

If the server handled one request at a time, a slow client (for example on a weak mobile connection) would block **everyone else**. With multiple threads, other users are served while one waits.

8.2 Why a *thread pool* and not "one new thread per request"?

One thread per request	Thread pool (what we use)
Creating and destroying a thread costs time and memory on every request	4 threads are created once at startup and reused forever

One thread per request	Thread pool (what we use)
10,000 visitors could create 10,000 threads and crash the machine	At most 4 threads; extra requests wait safely in the queue
Hard to control	Predictable, bounded resource use

8.3 The producer–consumer pattern

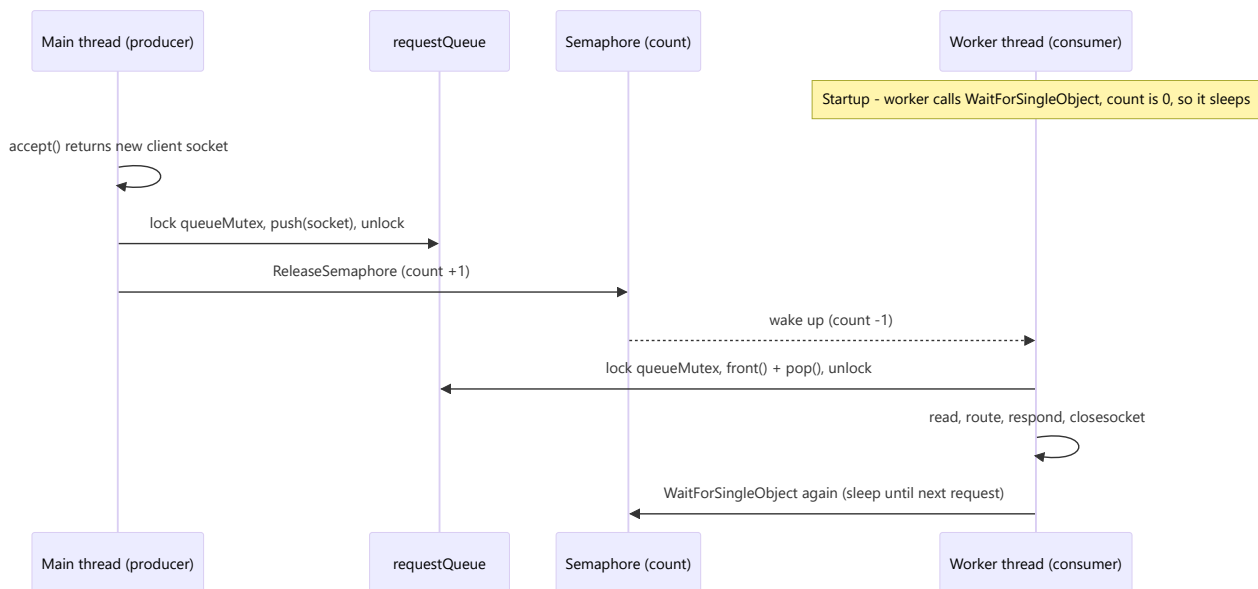


Diagram source: `diagrams/producer_consumer.mmd`

8.4 Why BOTH a semaphore and a mutex?

They solve **two different problems**:

Tool	Problem it solves	What goes wrong without it
Semaphore (<code>requestSemaphore</code>)	"Is there work?" It counts queued requests and puts idle workers to sleep.	Workers would have to spin in a loop (<code>while(queue.empty()){} </code>), burning 100% CPU doing nothing (busy waiting).
Mutex (<code>queueMutex</code>)	"Only one thread touches the queue at a time." <code>std::queue</code> is not thread-safe.	Two threads pushing or popping at the same moment can corrupt the queue's memory (a race condition), causing crashes or the same request being handled twice.

The semaphore guarantees that when a worker wakes up, **there is an item for it**, so the `front() + pop()` is safe.

8.5 `thread_local` : keeping users separate

```

thread_local string tlSessionId;    // whose session is this request?
thread_local string tlUserName;     // display name for the nav bar
thread_local string tlExtraHeaders; // e.g. Set-Cookie to send back

```

`thread_local` means **each thread has its own private copy** of these variables. Worker 1 may be serving Radhika while Worker 2 serves Shreya at the same moment, and each sees only its own user.

Why not just pass them as function parameters? About 40 page-building functions would all need extra parameters. `thread_local` keeps their signatures simple. Because a worker handles **one request at a time**, the values are always for the current request. The worker clears them at the start of every request (`tlSessionId.clear()` ...), so nothing leaks from the previous user.

8.6 `dbMutex` : protecting the database connection

There is **one** MySQL connection (`dbConn`) shared by all 4 workers. A MySQL connection **cannot be used by two threads at once**. If it were, their queries and results would get mixed up. So every function that talks to MySQL starts with:

```
lock_guard<mutex> lock(dbMutex);    // wait for our turn; auto-unlock on return
```

This makes every database operation a **critical section**.

Important to understand: because of `dbMutex`, **database work happens one request at a time**. The 4 threads still help, because receiving data, parsing, building HTML and sending responses all happen in parallel. Database access is the one shared step that is serialized. For higher throughput, the next step would be a **connection pool** (one connection per worker), letting MySQL's own row locks handle concurrency.

8.7 Deadlock safety

A deadlock needs two threads each holding a lock the other wants. Here: - `queueMutex` is held only for a push or a pop, never while doing anything else. - `dbMutex` is held only inside DB functions, and the code never takes `queueMutex` while holding `dbMutex`. - `lock_guard` always releases the lock, even on early `return`.

So there is **no circular wait**, and deadlock cannot occur between these two locks.

9. Life of one request

Example: a logged-in user clicks **"Add to Cart"** for product 3, quantity 2.

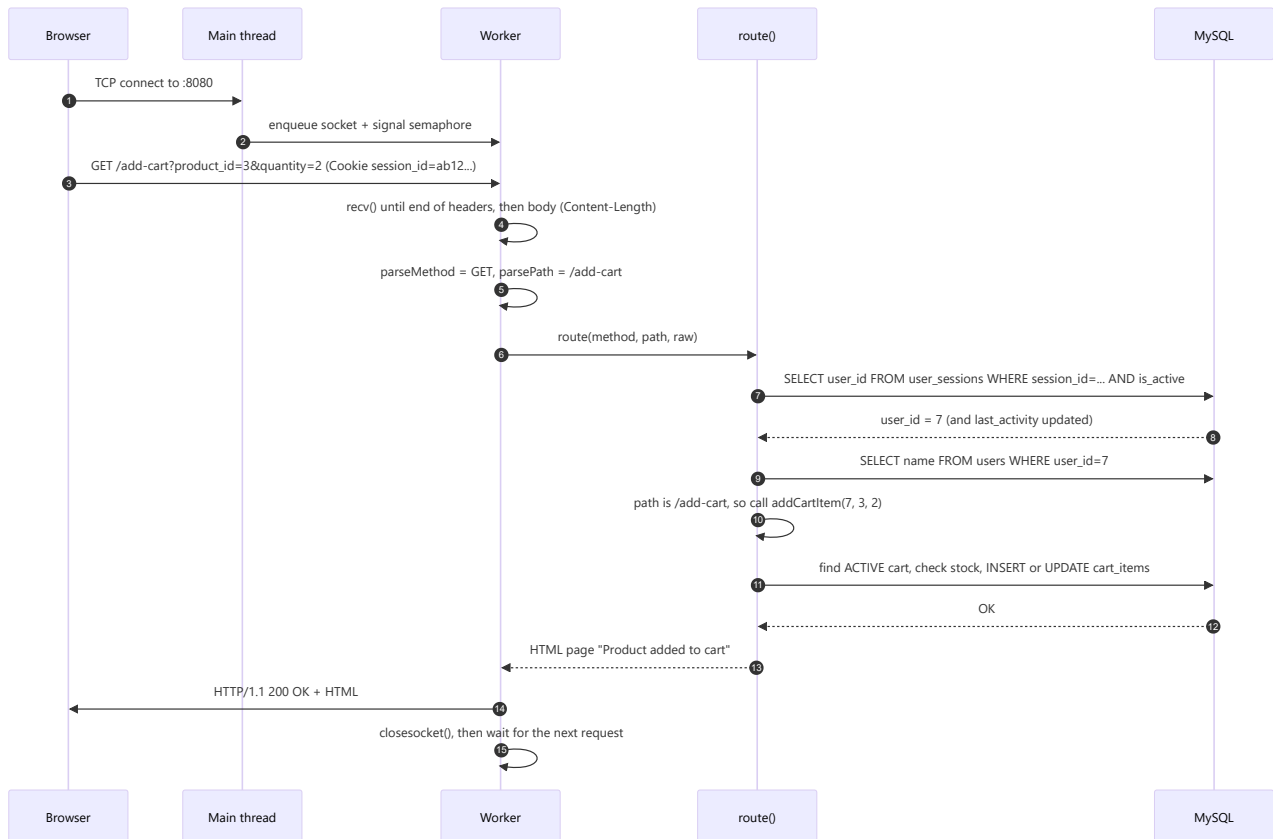


Diagram source: [diagrams/request_lifecycle.mmd](#)

10. Understanding HTTP: what the server actually reads

A browser sends **plain text** over TCP. For example, a login form submission looks like this:

```
POST /login HTTP/1.1\r\n
Host: localhost:8080\r\n
Cookie: session_id=ab12cd34...\r\n
Content-Type: application/x-www-form-urlencoded\r\n
Content-Length: 41\r\n
\r\n
email=radhika%40mail.com&password=secret1
```

Part	Parsed by	How
Method (<code>POST</code>)	<code>parseMethod()</code>	First word of the request
Path (<code>/login</code>)	<code>parsePath()</code>	Second word, with anything after <code>?</code> removed
Query string (<code>?product_id=3&quantity=2</code>)	<code>parseQuery()</code>	Text after <code>?</code> , split on <code>&</code> and <code>=</code>
Headers end	<code>worker()</code>	Looks for the blank line <code>\r\n\r\n</code>
Body length	<code>contentLength()</code>	Reads the <code>Content-Length:</code> header (case-insensitive)

Part	Parsed by	How
Form body	<code>parseFormBody()</code>	Text after <code>\r\n\r\n</code> , split on <code>&</code> and <code>=</code>
Special characters	<code>urlDecode()</code>	<code>%40</code> → <code>@</code> , <code>+</code> → space
Cookie	<code>getCookie()</code>	Finds the <code>Cookie:</code> header and the <code>session_id=</code> pair

Why read in a loop? TCP is a **stream**: one `recv()` may return only part of the request. The worker keeps calling `recv()` until it has the full headers (`\r\n\r\n`), then keeps reading until it has `Content-Length` bytes of body. Requests with headers over 64 KB are dropped, which protects memory.

The response is also plain text, built by `makeHttpResponse()`:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 1834
Set-Cookie: session_id=ab12...; Path=/; HttpOnly; SameSite=Lax    ← only when logging in or out
Connection: close

<html>...</html>
```

`Connection: close` means one request per connection. This is simpler, because the worker closes the socket and is free for the next one.

11. The router

`route()` is the **traffic controller**. It runs two steps for every request.

Step 1: identify the user (every request). It reads the `session_id` cookie and looks it up in `user_sessions`. If found, it fills `t1sessionId` and `t1UserName`, so the page shows "Hi, Radhika" and the logged-in menu.

Step 2: match the path:

URL	Method	Login?	Handler	Does
<code>/</code>	GET	no	inline	Home page; different buttons for guests and users
<code>/products</code>	GET	no	<code>buildProductsPage()</code>	List all products
<code>/signup</code>	GET / POST	no	<code>signupForm()</code> / <code>handleSignup()</code>	Show form / create account
<code>/login</code>	GET / POST	no	<code>loginForm()</code> / <code>handleLogin()</code>	Show form / log in, create session
<code>/logout</code>	GET	—	<code>logoutSession()</code>	End session, delete cookie
<code>/session</code>	GET	yes	<code>buildSessionPage()</code>	Show your login sessions
<code>/cart</code>	GET	yes	<code>buildCartPage()</code>	Show cart and total

URL	Method	Login?	Handler	Does
/add-cart	GET	yes	addCartItem()	Add or increase an item
/remove-cart	GET	yes	removeCartItem()	Remove an item
/checkout	GET	yes	checkoutCart()	Place order (transaction)
/orders	GET	yes	buildOrdersPage()	Order and payment history
anything else	—	—	pageWrapper("404")	Page not found (HTTP 404)

For "login required" pages, if the session is not valid the router returns a **"Please log in first"** page instead of running the handler. This is the access-control check.

12. Building pages (HTML)

Every page goes through `pageWrapper(title, body)`, which adds: - the shared **CSS** (styles), - the **navigation bar**: Cart, Orders, Session and Logout when logged in; Login and Sign Up buttons when not, - the **login and sign-up popups** (modals) plus a small JavaScript to open and close them.

Why one wrapper? Every page looks consistent, and the layout is defined in one place (the DRY principle: Don't Repeat Yourself).

Why `htmlEscape()` ? Any text that came from users or the DB (names, product descriptions) is escaped: `<` becomes `<`, `>` becomes `>`, and so on. Without this, a user could register with the name `<script>...</script>` and that script would run in other people's browsers. That attack is **Cross-Site Scripting (XSS)**.

13. Users: signup and login

Signup (`POST /signup` → `handleSignup()`)

1. Check that name, email and password are not empty.
2. Lock `dbMutex`.
3. `SELECT user_id FROM users WHERE email = '...'`. If a row exists, show "Email already registered".
4. `INSERT INTO users (name, email, password)`.
5. Show the login form with "Signup successful".

Why check the email first when the column is already `UNIQUE` ? For a friendly error message. The `UNIQUE` constraint is still the real guarantee.

Why `esc()` around every string? `esc()` calls `mysql_real_escape_string`, which neutralises quotes. Without it, an email like `' OR '1'='1` could change the meaning of the SQL query. That attack is **SQL injection**.

Login (`POST /login` → `handleLogin()`)

1. Look up the user by email (`SELECT user_id, name, password`).
2. Compare passwords. Wrong → "Incorrect password".
3. Create a session (next section) and send the cookie.

14. Sessions: how the server remembers you

14.1 The problem

HTTP is stateless. Every request is independent. The server has no built-in way to know that the request for `/cart` comes from the same person who logged in a minute ago. So we need a **session**: a random ticket the browser shows on every request.

14.2 Where sessions live: the `user_sessions` table

Column	Meaning
<code>session_id</code>	The random ticket (32 hex characters)
<code>user_id</code>	Which user it belongs to
<code>login_time</code>	When they logged in
<code>last_activity</code>	When they last made a request (updated every request)
<code>logout_time</code>	When they logged out (<code>NULL</code> while active)
<code>is_active</code>	<code>TRUE</code> = valid, <code>FALSE</code> = logged out

Why store sessions in the database instead of in server memory? - They **survive a server restart**. Users stay logged in. - They give a **login history** (the "My Session" page) for auditing. - **Multiple devices** are supported naturally: each login is its own row. - Logout just flips `is_active`, so old sessions remain as a record.

14.3 The full session lifecycle

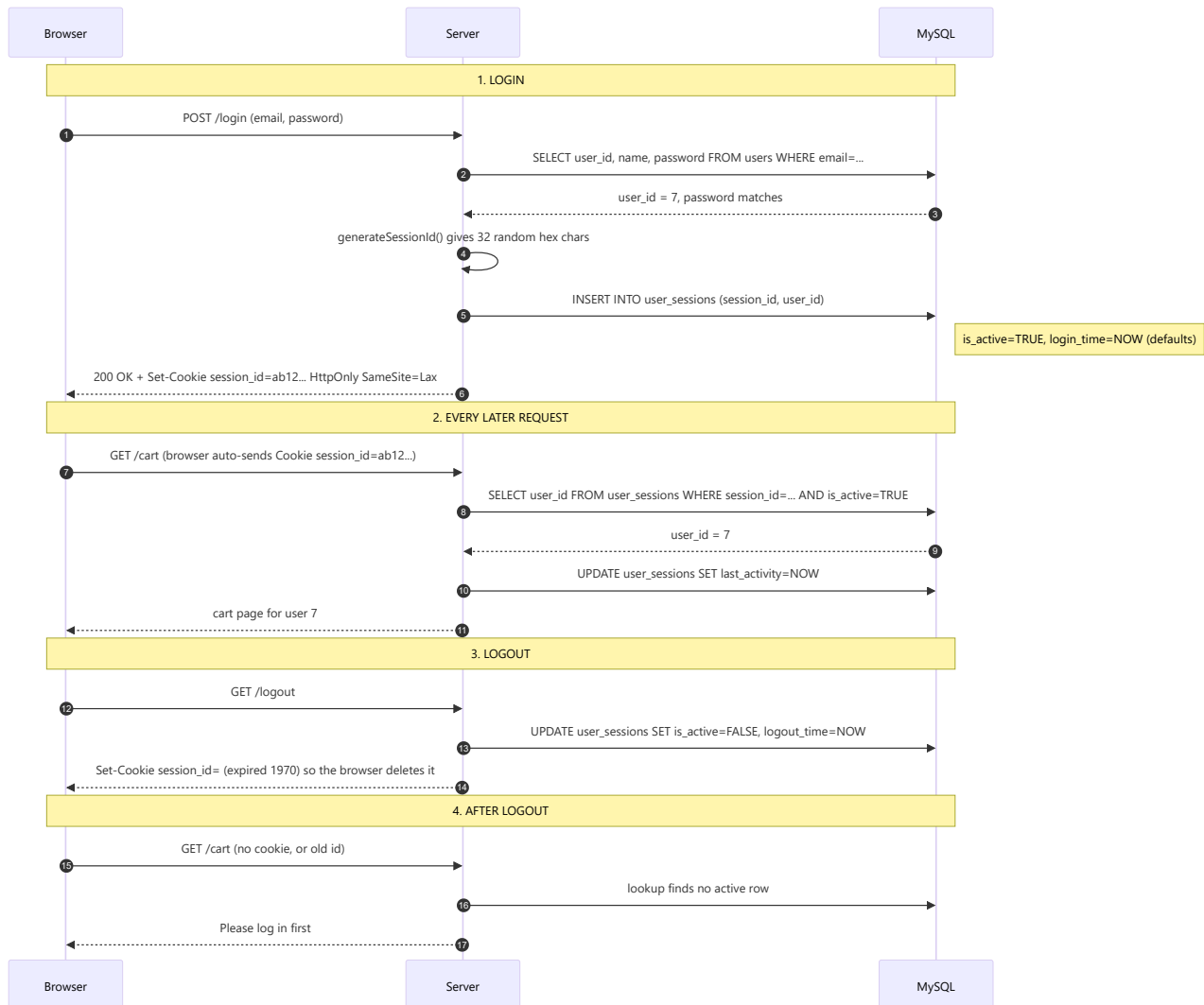


Diagram source: diagrams/session_lifecycle.mmd

14.4 Each step explained

a) Creating the ID: `generateSessionId()` - Builds 32 characters from `0-9a-f` using a random generator (`mt19937_64`) seeded by `random_device`. - 32 hex characters = **128 bits**, about 3.4×10^{38} possibilities. Guessing a valid one is practically impossible. - **Why random and not just 1, 2, 3...?** With sequential IDs, anyone could change their cookie to `2` and become another user.

b) Saving it: `createSession()` - `INSERT INTO user_sessions (session_id, user_id)`. The database fills in `login_time`, `last_activity` and `is_active` from column defaults. - If the ID ever collides with an existing one (primary key violation), it retries, up to 5 times.

c) Giving it to the browser: the cookie

```
Set-Cookie: session_id=ab12...; Path=/; HttpOnly; SameSite=Lax
```

Attribute	Meaning	Why
Path=/ 	Send the cookie for every URL on the site	All pages need to know the user
HttpOnly	JavaScript cannot read it	Even if an XSS bug existed, a script could not steal the session
SameSite=Lax	Not sent with cross-site POSTs or background requests	Reduces Cross-Site Request Forgery (CSRF)

Why a cookie instead of putting the ID in every link? The browser attaches cookies **automatically**, so no page has to remember to add `?session_id=...`. IDs in URLs also leak into browser history and server logs. (The code still *accepts* `?session_id=` as a fallback for old links.)

d) Checking it on every request: `getSessionUserId()` - Looks for the ID **and** `is_active = TRUE`. A logged-out session is useless even if someone still has the cookie. - Updates `last_activity`, which shows when the session was last used.

e) Ending it: `logoutSession()` - Sets `is_active = FALSE` and records `logout_time`. The row is **kept**, not deleted, for history. - Sends an expired cookie so the browser forgets it.

f) Multiple sessions

```
Radhika → Chrome → session A123 (active)
Radhika → Edge   → session B456 (active)
Shreya  → Chrome → session C789 (active)
```

Logging out in Chrome only ends A123. Edge stays logged in. The **My Session** page (`/session`) lists your last 20 sessions and highlights the one this browser is using.

15. Products page

`/products` → `buildProductsPage()`: 1. `SELECT product_id, name, description, price, stock, ... FROM products`. 2. Builds an HTML table. For each product: - `stock = 0` → "Out of stock" - not logged in → a "Login to buy" button that opens the login popup - logged in → a quantity box (`max = stock`) and an **Add to Cart** button

Why show different buttons? It guides the user and avoids errors before they happen. The server still re-checks everything, because the browser cannot be trusted.

⚠ Known bug: the query selects a column named `category`, but the table has `category_id`, so MySQL returns *Unknown column*. Fix: join the `categories` table (see §21).

16. Cart

Data model

- `cart` : one row per cart, with `status` = `ACTIVE` or `CHECKED_OUT` .
- `cart_items` : one row per product in the cart, with its `quantity` .
- A user has **at most one ACTIVE cart**. Old carts stay as `CHECKED_OUT` history.

`getActiveCartIdLocked(userId, createIfMissing)`

Finds the user's `ACTIVE` cart. If there is none and `createIfMissing` is true, it inserts a new one. The name ends in "**Locked**" to warn programmers that **the caller must already hold** `dbMutex` . This makes "find, else create" one atomic step, so two requests can't create two active carts.

Add to cart (`/add-cart` → `addCartItem()`)

1. Reject quantity ≤ 0 .
2. Get or create the active cart.
3. Read the product's `stock` .
4. Read how many are **already** in the cart.
5. If `already + new > stock` → "Requested quantity exceeds available stock".
6. Already in the cart → `UPDATE quantity = quantity + n` . Otherwise → `INSERT` a new row.

Why update instead of inserting twice? The table has `UNIQUE(cart_id, product_id)` , so one product appears once per cart with a total quantity. This is cleaner and makes totals easy.

Why check stock here *and* again at checkout? Here it gives early, friendly feedback. At checkout it is the **real** guarantee, because stock may have changed in between.

View cart (`/cart`) and remove (`/remove-cart`)

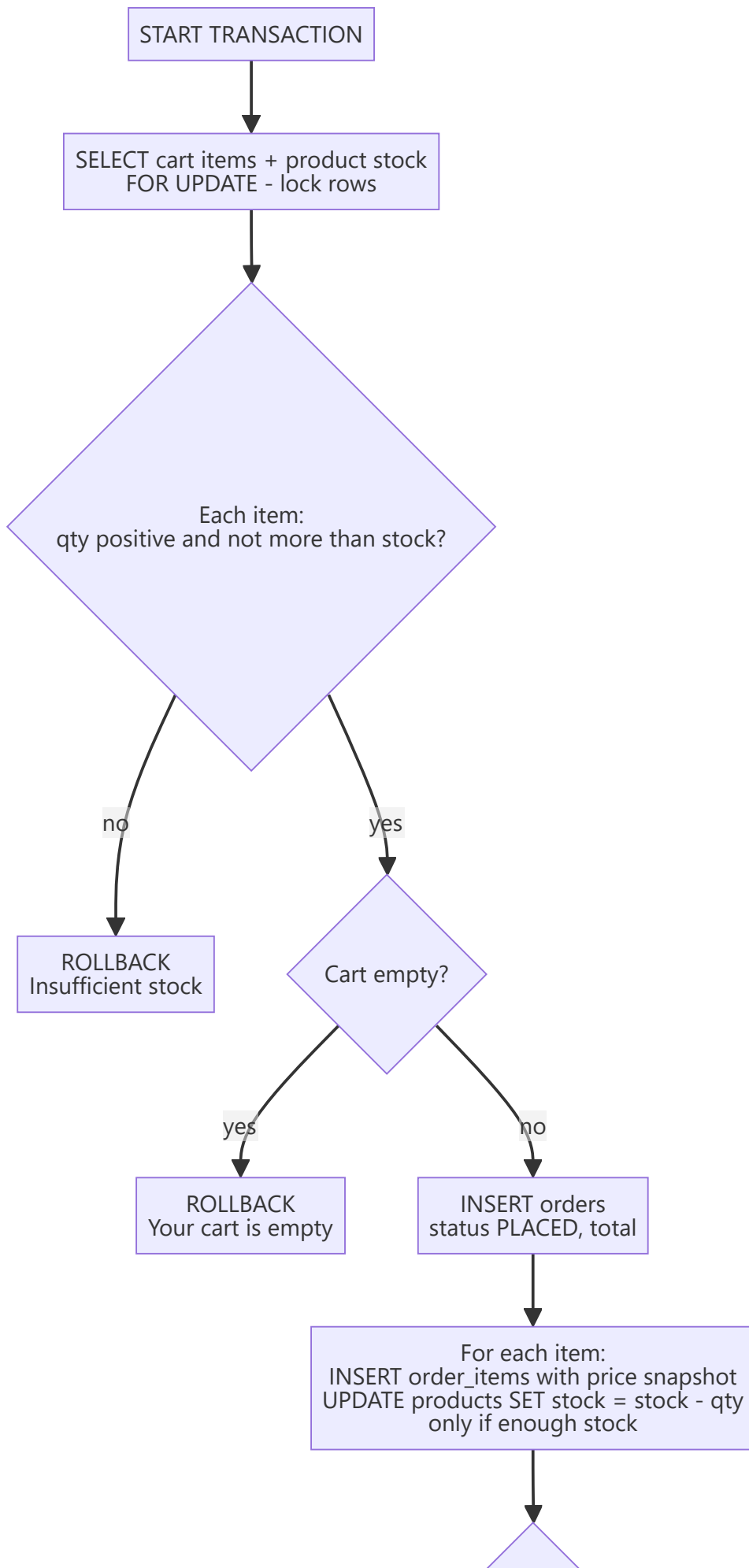
- The view joins `cart_items` with `products` to show name, price and `subtotal = quantity × price` , plus a grand total.
- Remove runs `DELETE FROM cart_items WHERE cart_id=... AND product_id=...` .

17. Checkout: the transaction

This is the **DBMS heart** of the project: `/checkout` → `checkoutCart()` .

17.1 What has to happen

Placing an order touches **five tables**:



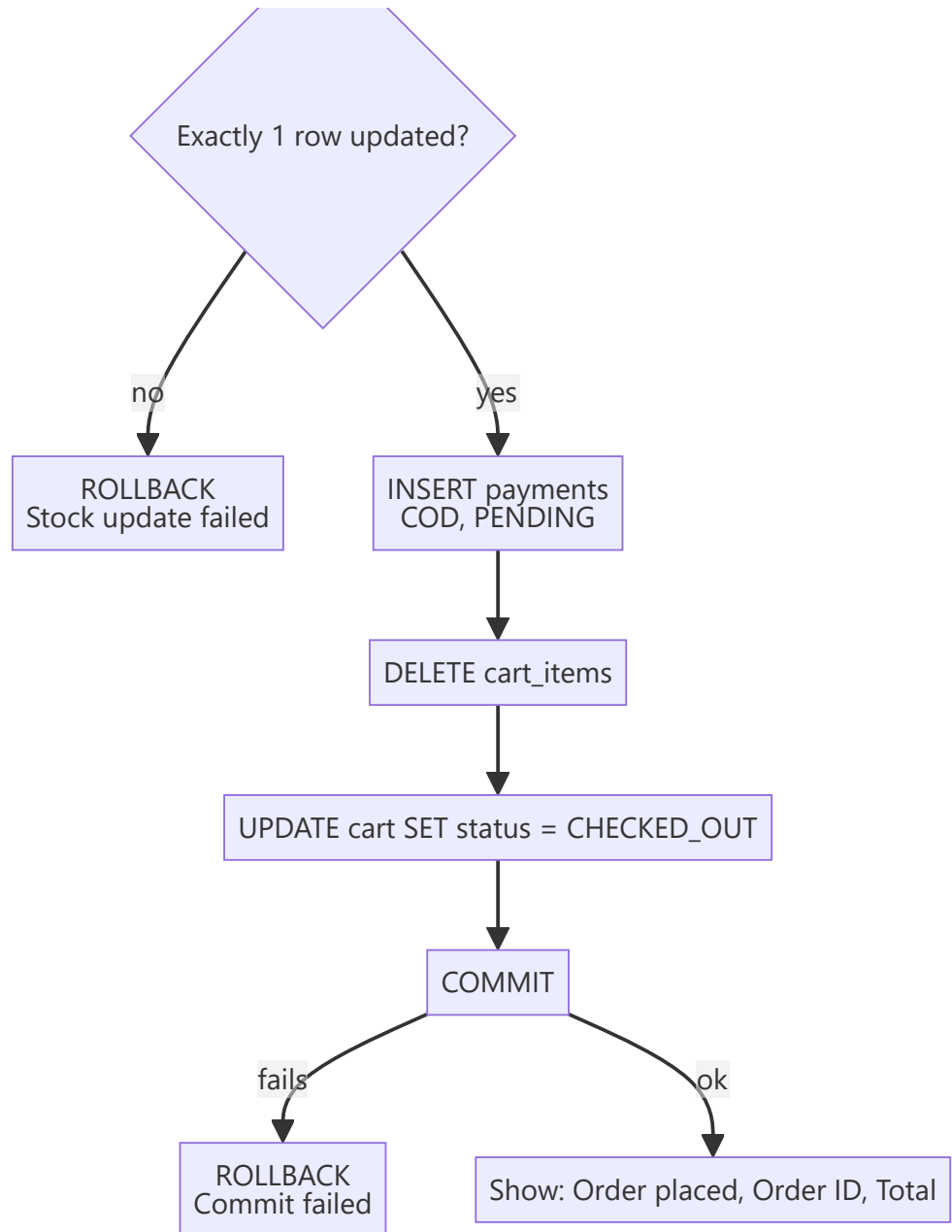


Diagram source: diagrams/checkout_flow.mmd

17.2 Why a transaction? (ACID explained with this example)

Suppose the server crashes right after creating the order but **before** reducing stock. Without a transaction, there is an order for items that were never deducted, and the shop could sell them twice. A transaction prevents that:

ACID property	Meaning	In our checkout
A tomicity	All or nothing	Order, items, stock change, payment, and cart clear all succeed, or ROLLBACK undoes every one
C onsistency	Rules are never broken	Stock never goes negative (<code>WHERE stock >= qty</code>); foreign keys and <code>UNIQUE(cart_id)</code> hold
I solation	Concurrent transactions don't interfere	<code>FOR UPDATE</code> locks the rows; <code>dbMutex</code> serializes access

ACID property	Meaning	In our checkout
Durability	Once committed, it stays	After <code>COMMIT</code> , InnoDB writes to its log, so the order survives a crash

17.3 Preventing overselling: three layers of protection

Imagine **2 users buy the last 1 unit** at the same moment:

1. `dbMutex` : only one checkout runs on the connection at a time, so they are processed one after the other.
2. `SELECT ... FOR UPDATE` : locks the product rows until `COMMIT` or `ROLLBACK`. Any other transaction (even from another program) must wait.
3. `UPDATE ... SET stock = stock - qty WHERE stock >= qty` , followed by checking `mysql_affected_rows == 1` : the update only succeeds if enough stock exists **at that instant**. Otherwise the whole order is rolled back.

Result: the first user gets the item. The second sees "*Insufficient stock*" and nothing is half-saved.

Why three layers if one would do? Defence in depth. Layer 1 works today because there is one connection. Layers 2 and 3 keep the code correct if we later add a connection pool, or if another program (an admin panel, for example) changes stock directly.

17.4 Other design decisions

Decision	Why
<code>order_items.price</code> stores the price at purchase time	If the product price changes tomorrow, old orders still show what the customer actually paid.
<code>orders.cart_id</code> is <code>UNIQUE</code>	One cart can become at most one order. This blocks accidental double-orders from the same cart.
<code>payments.order_id</code> is <code>UNIQUE</code>	One payment record per order.
Cart becomes <code>CHECKED_OUT</code> instead of being deleted	Keeps history. The next "Add to cart" automatically creates a fresh <code>ACTIVE</code> cart.
Payment is <code>COD</code> / <code>PENDING</code>	No real payment gateway; cash on delivery is recorded as pending.

18. Orders page

/orders → buildOrdersPage() :

```
SELECT o.order_id, o.total, o.status, o.created_at, p.method, p.status
FROM orders o
LEFT JOIN payments p ON o.order_id = p.order_id
WHERE o.user_id = ?
ORDER BY o.created_at DESC;
```

Why LEFT JOIN ? An order should still appear even if, for some reason, it has no payment row. An **INNER JOIN** would silently hide it.

19. Database design and ER diagrams

Nine tables. This section has the full ER diagrams, data dictionary and design notes.

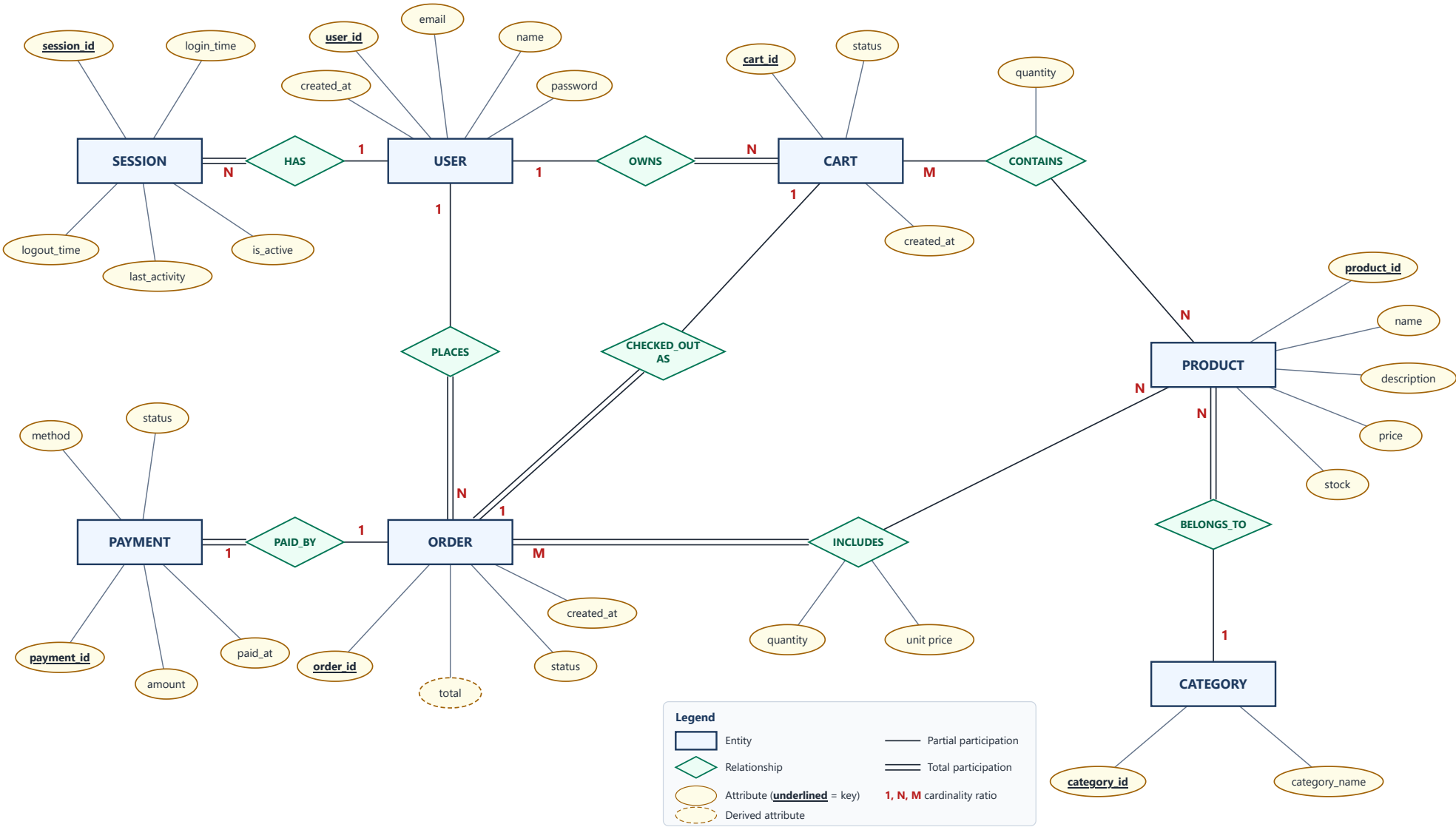
```
users └─< user_sessions (1 user : many sessions)
      │
      └─< cart └─< cart_items >─┐
          │                     │
          │                     └─ 0..1 ─ orders │ (1 cart : at most 1 order)
          │
          └─< orders └─< order_items >─ products >─ categories
              │
              └─ 0..1 ─ payments
```

Concept	Where you see it	Why
Primary keys (<code>AUTO_INCREMENT</code>)	Every table	Unique, stable identity for each row
Foreign keys	e.g. <code>cart_items.product_id</code> → <code>products</code>	The DB refuses rows that point to non-existent data (referential integrity)
UNIQUE constraints	<code>users.email</code> , <code>(cart_id, product_id)</code> , <code>orders.cart_id</code> , <code>payments.order_id</code>	Business rules enforced by the database itself
Indexes	<code>user_sessions(user_id)</code> , <code>(is_active)</code> , <code>(login_time)</code>	The session lookup runs on every request, so it must be fast
Defaults	<code>created_at</code> , <code>status</code> , <code>is_active</code>	Less code; the DB fills in sensible values
Normalization (3NF)	Categories in their own table; junction tables for M:N	No duplicated data, so no update anomalies
Transactions	Checkout	ACID (see §17)

19.1 Conceptual ER diagram (Chen notation)

ecommerce_db — Conceptual ER Diagram (Chen Notation)

Entities, attributes, relationships, cardinality ratios and participation constraints



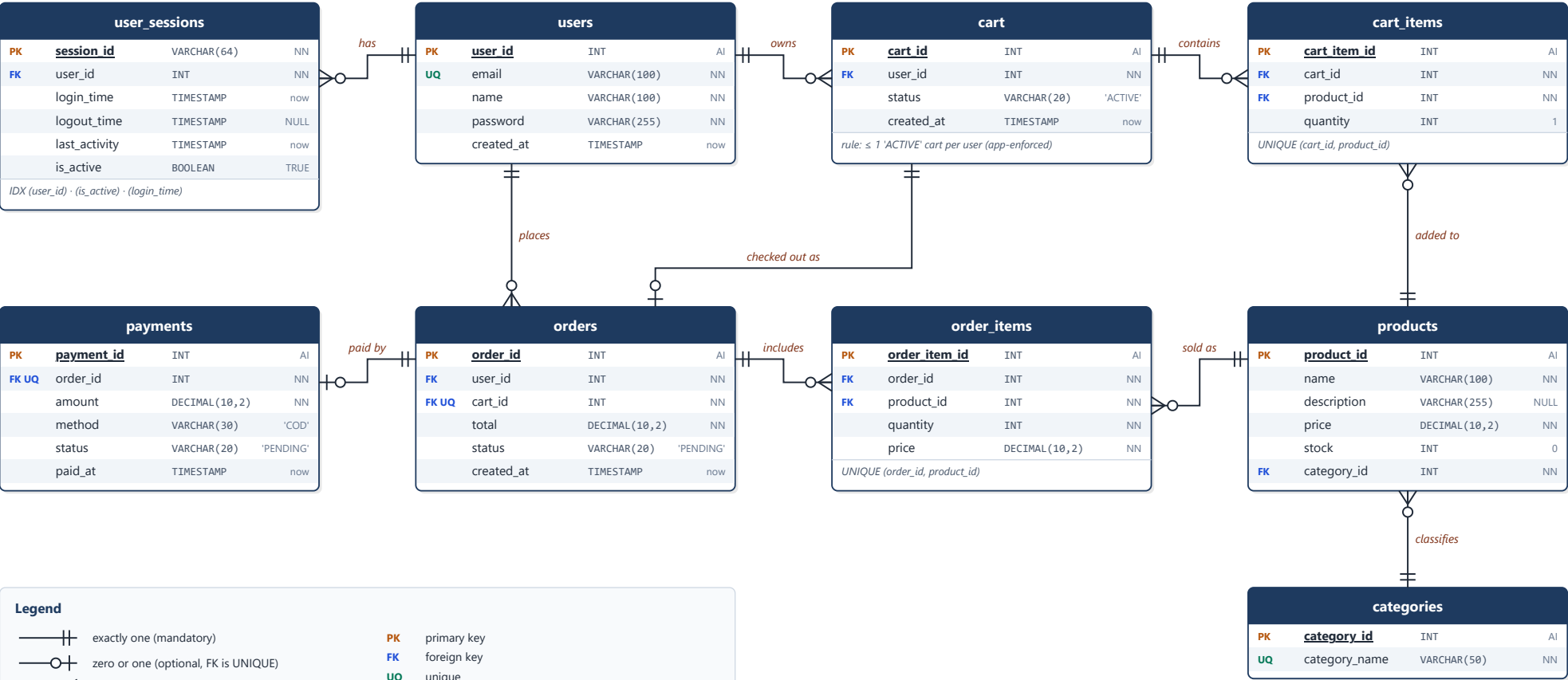
How to read it:

- **Rectangles** are entities, **diamonds** are relationships, **ovals** are attributes.
- **Underlined** attributes are keys. The **dashed** oval (`ORDER.total`) is a *derived* attribute. It can be computed as $\Sigma \text{quantity} \times \text{unit price}$ over the order's items but is stored so it does not have to be recomputed.
- **1 / N / M** are cardinality ratios. A **double line** means *total participation*: every instance of that entity must take part in the relationship. For example, every ORDER is placed by a USER.
- Foreign keys are **not** drawn as attributes here. In a conceptual model they are represented by the relationships themselves.
- `cart_items` and `order_items` appear as the **M:N relationships** **CONTAINS** **and** **INCLUDES** , with their own attributes (`quantity` , `unit price`). They become separate tables only in the relational model (§19.2).

19.2 Relational ER diagram (crow's foot)

ecommerce_db — Relational ER Diagram (Crow's Foot)

9 tables · 10 foreign-key relationships · generated from ecommerce_db.sql



Each connector runs from the parent's primary key to the child's foreign-key row. The symbol at each end gives the *minimum* and *maximum* number of rows on that side.

19.3 Entities (data dictionary)

Legend: **PK** primary key · **FK** foreign key · **UQ** unique · **AI** auto-increment · **NN** not null.

19.3.1 `users` : registered customers

Column	Type	Constraints	Notes
<code>user_id</code>	INT	PK, AI	Surrogate key
<code>email</code>	VARCHAR(100)	NN, UQ	Login identifier; checked for duplicates at registration
<code>name</code>	VARCHAR(100)	NN	Display name
<code>password</code>	VARCHAR(255)	NN	Currently stored as plain text (see §19.7)
<code>created_at</code>	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	

19.3.2 `user_sessions` : login sessions (one row per login)

Column	Type	Constraints	Notes
<code>session_id</code>	VARCHAR(64)	PK	Random token, sent to the browser as a cookie
<code>user_id</code>	INT	FK → <code>users</code> , NN, indexed	
<code>login_time</code>	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP, indexed	
<code>logout_time</code>	TIMESTAMP	NULL	Set on logout
<code>last_activity</code>	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Updated on each authenticated request
<code>is_active</code>	BOOLEAN	NN, DEFAULT TRUE, indexed	Set to FALSE on logout

Indexes: `idx_sessions_user` (`user_id`) , `idx_sessions_active` (`is_active`) , `idx_sessions_login` (`login_time`) .

19.3.3 `categories` : product classification

Column	Type	Constraints	Notes
<code>category_id</code>	INT	PK, AI	
<code>category_name</code>	VARCHAR(50)	NN, UQ	15 seeded values (Electronics, Audio, Books, ...)

19.3.4 `products` : catalogue and inventory

Column	Type	Constraints	Notes
<code>product_id</code>	INT	PK, AI	

Column	Type	Constraints	Notes
name	VARCHAR(100)	NN	
description	VARCHAR(255)	NULL	
price	DECIMAL(10,2)	NN	Current selling price
stock	INT	NN, DEFAULT 0	Decrementated atomically at checkout
category_id	INT	FK → categories , NN	

19.3.5 cart : shopping carts (history kept)

Column	Type	Constraints	Notes
cart_id	INT	PK, AI	
user_id	INT	FK → users , NN	
status	VARCHAR(20)	NN, DEFAULT 'ACTIVE'	ACTIVE → CHECKED_OUT
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	

19.3.6 cart_items : products in a cart (resolves CART M:N PRODUCT)

Column	Type	Constraints	Notes
cart_item_id	INT	PK, AI	
cart_id	INT	FK → cart , NN	
product_id	INT	FK → products , NN	
quantity	INT	NN, DEFAULT 1	Incremented when the same product is added again

Composite unique key: (cart_id, product_id) . A product appears at most once per cart.

19.3.7 orders : placed orders

Column	Type	Constraints	Notes
order_id	INT	PK, AI	
user_id	INT	FK → users , NN	
cart_id	INT	FK → cart , NN, UQ	One order per cart
total	DECIMAL(10,2)	NN	Derived: $\sum \text{order_items.quantity} \times \text{price}$
status	VARCHAR(20)	DEFAULT 'PENDING'	Application inserts 'PLACED'
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	

19.3.8 order_items : products in an order (resolves ORDER M:N PRODUCT)

Column	Type	Constraints	Notes
order_item_id	INT	PK, AI	
order_id	INT	FK → orders , NN	

Column	Type	Constraints	Notes
product_id	INT	FK → products , NN	
quantity	INT	NN	
price	DECIMAL(10,2)	NN	Snapshot of the unit price at purchase time

Composite unique key: (order_id, product_id) .

19.3.9 payments : payment record per order

Column	Type	Constraints	Notes
payment_id	INT	PK, AI	
order_id	INT	FK → orders , NN, UQ	One payment per order
amount	DECIMAL(10,2)	NN	Equals orders.total at creation
method	VARCHAR(30)	DEFAULT 'COD'	
status	VARCHAR(20)	DEFAULT 'PENDING'	
paid_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	

19.4 Relationships

#	Parent (1)	Child	Relationship	Cardinality	Implemented by	Participation
R1	users	user_sessions	has	1 : 0..N	user_sessions.user_id NN	Session total, user partial
R2	users	cart	owns	1 : 0..N	cart.user_id NN	Cart total, user partial
R3	users	orders	places	1 : 0..N	orders.user_id NN	Order total, user partial
R4	categories	products	classifies	1 : 0..N	products.category_id NN	Product total, category partial
R5	cart	cart_items	contains	1 : 0..N	cart_items.cart_id NN	
R6	products	cart_items	added to	1 : 0..N	cart_items.product_id NN	
R7	cart	orders	checked out as	1 : 0..1	orders.cart_id NN UNIQUE	Order total, cart partial
R8	orders	order_items	includes	1 : 0..N (app: 1..N)	order_items.order_id NN	
R9	products	order_items	sold as	1 : 0..N	order_items.product_id NN	
R10	orders	payments	paid by	1 : 0..1	payments.order_id NN UNIQUE	Payment total, order partial

R5 + R6 together resolve the **M:N** relationship *CART contains PRODUCT*. R8 + R9 resolve the **M:N** relationship *ORDER includes PRODUCT*. The constraint is what turns R7 and R10 into one-to-one relationships: the **UNIQUE** on the foreign key. Without it they would be one-to-many.

None of the foreign keys declare `ON DELETE / ON UPDATE` actions, so MySQL uses the default `RESTRICT`. A parent row (for example a user who has orders) cannot be deleted while child rows still reference it.

19.5 Conceptual → relational mapping

Conceptual element	Mapping rule	Resulting table / column
Strong entities USER, SESSION, CATEGORY, PRODUCT, CART, ORDER, PAYMENT	One table each, key → PK	<code>users</code> , <code>user_sessions</code> , <code>categories</code> , <code>products</code> , <code>cart</code> , <code>orders</code> , <code>payments</code>
1:N HAS, OWNS, PLACES, BELONGS_TO	FK on the N side	<code>user_sessions.user_id</code> , <code>cart.user_id</code> , <code>orders.user_id</code> , <code>products.category_id</code>
1:1 CHECKED_OUT_AS (ORDER total)	FK + UNIQUE on the total-participation side	<code>orders.cart_id</code> UNIQUE
1:1 PAID_BY (PAYMENT total)	FK + UNIQUE on the total-participation side	<code>payments.order_id</code> UNIQUE
M:N CONTAINS {quantity}	New table with both FKs + relationship attributes	<code>cart_items(cart_id, product_id, quantity)</code>
M:N INCLUDES {quantity, unit price}	New table with both FKs + relationship attributes	<code>order_items(order_id, product_id, quantity, price)</code>

Both junction tables use a surrogate PK (`cart_item_id`, `order_item_id`). The natural key (`fk1`, `fk2`) is still enforced by a `UNIQUE` constraint, so the M:N semantics hold.

19.6 Data lifecycle: the checkout transaction

Covered in full in [§17 Checkout: the transaction](#).

19.7 Design notes and known limitations

These come from reading the schema and code. They are worth discussing when the design is reviewed.

1. **"One ACTIVE cart per user" is enforced only by the application.** Today it holds because every DB call runs under the single global `dbMutex`, so the "find active cart, else insert" step never interleaves. If the server moved to a connection pool, two requests could each create an `ACTIVE` cart. The database could enforce the rule itself in MySQL 8 with a generated column, `active_user_id = IF(status='ACTIVE', user_id, NULL)`, and a `UNIQUE` index on it.
2. **Redundant path** `orders.user_id`. The user can already be reached through `orders.cart_id → cart.user_id`. No constraint keeps the two consistent. This is not a 3NF violation, because `cart_id` is a candidate key of `orders`. It is still an inter-table redundancy, kept for simpler and faster "my orders" queries.
3. **Stored derived values.** `orders.total` and `payments.amount` can both be computed from `order_items`. Storing them is acceptable because they are written once inside the checkout transaction and never changed afterwards.

4. **order_items.price is deliberate, not redundant.** It records the price at purchase time, so a later change to `products.price` does not rewrite order history.
5. **Status columns are free-text VARCHAR.** `cart.status`, `orders.status` and `payments.status` accept any string. Adding `ENUM` or `CHECK` constraints would block invalid states. The `orders.status` default (`'PENDING'`) also differs from the value the application inserts (`'PLACED'`).
6. **No CHECK on quantities or money.** `stock >= 0`, `quantity > 0` and `price >= 0` are only guarded by application logic.
7. **Passwords are stored in plain text.** They should be stored as a salted hash, for example bcrypt or Argon2. The `VARCHAR(255)` column is already wide enough.
8. **Schema / code mismatch.** `buildProductsPage()` in `main.cpp` selects a column named `category` from `products`, but the table has `category_id`. A `JOIN categories` is needed to show the category name.

20. OS and DBMS concepts used (syllabus map)

Concept	Subject	Where in code
Threads	OS	<code>std::thread</code> , <code>worker()</code>
Thread pool	OS	4 workers created in <code>main()</code>
Producer-consumer	OS	main thread ↔ <code>requestQueue</code> ↔ workers
Counting semaphore	OS	<code>CreateSemaphore</code> , <code>WaitForSingleObject</code> , <code>ReleaseSemaphore</code>
Mutex / mutual exclusion	OS	<code>queueMutex</code> , <code>dbMutex</code> , <code>lock_guard</code>
Critical section	OS	Every block inside a <code>lock_guard</code>
Race condition (prevented)	OS	Queue access; stock update
Busy waiting (avoided)	OS	Semaphore makes idle workers sleep
Deadlock (avoided)	OS	No nested lock acquisition (§8.7)
Thread-local storage	OS	<code>thread_local</code> request context
Sockets / TCP / IPC	OS / Networks	<code>socket</code> , <code>bind</code> , <code>listen</code> , <code>accept</code> , <code>recv</code> , <code>send</code>
ER modelling	DBMS	9 entities, 10 relationships
Normalization	DBMS	3NF schema
Keys and constraints	DBMS	PK, FK, UNIQUE, NOT NULL, DEFAULT
Indexing	DBMS	<code>user_sessions</code> indexes
Joins	DBMS	Cart page, session page, orders page (<code>LEFT JOIN</code>)
Transactions / ACID	DBMS	<code>checkoutCart()</code>
Locking / concurrency control	DBMS	<code>SELECT ... FOR UPDATE</code>
Commit / rollback	DBMS	<code>mysql_commit</code> , <code>mysql_rollback</code>
SQL injection prevention	Security	<code>esc()</code>
XSS prevention	Security	<code>htmlEscape()</code>
Session management	Web	<code>user_sessions</code> + cookie

21. Known limitations and how to improve them

Knowing the weaknesses of your own design is part of understanding it.

#	Limitation	Why it matters	Improvement
1	Single DB connection + global <code>dbMutex</code>	All DB work is serialized, which limits throughput under load	Connection pool (one <code>MYSQL*</code> per worker)
2	Passwords stored in plain text	A DB leak exposes every password	Store a salted hash (bcrypt / Argon2) and compare hashes
3	Checkout / logout / add-cart use GET links	<code>SameSite=Lax</code> still sends the cookie when following a link from another site, so a malicious page could trigger <code>/checkout</code> (CSRF)	Use POST forms + a CSRF token
4	Session ID appears in some URLs (<code>?session_id=</code>)	Leaks into history and logs	Rely on the cookie only; remove the URL fallback
5	Sessions never expire	A stolen cookie works forever	Reject sessions with <code>last_activity</code> older than e.g. 30 minutes
6	<code>mt19937_64</code> is not cryptographically secure	Its output can be predicted in theory	Use <code>BCryptGenRandom</code> (Windows CSPRNG)
7	Products query uses a missing <code>category</code> column	Products page shows a DB error	<code>SELECT p..., c.category_name FROM products p JOIN categories c USING(category_id)</code>
8	DB password hard-coded in <code>main.cpp</code>	Exposed in the public repo	Read from an environment variable or config file not committed to git
9	Semaphore maximum is 1000	Beyond 1000 queued requests, <code>ReleaseSemaphore</code> fails and that request is never served	Bound the queue and return HTTP 503 when full
10	Session looked up twice on protected pages	Two extra queries per request	Reuse the user ID found at the top of <code>route()</code>
11	Windows-only (<code>winsock2</code> , <code>CreateSemaphore</code>)	Won't compile on Linux	Use <code>std::counting_semaphore</code> (C++20) and POSIX sockets, or a portability layer
12	No <code>Keep-Alive</code>	A new TCP connection for every request	Support persistent connections

22. Viva / review questions

Q1. Why use a thread pool instead of creating a thread for every request? Creating threads is expensive, and the number of threads would be unbounded. A pool reuses a fixed number of threads, and extra work waits in a queue.

Q2. Why do you need both a semaphore and a mutex? The semaphore answers "is there work?" and lets idle threads sleep. The mutex answers "who may touch the queue right now?" and prevents corruption.

Q3. What is a race condition? Show one you prevented. When the result depends on the timing of threads. Example: two users buying the last unit. Prevented by `dbMutex`, `FOR UPDATE`, and `WHERE stock >= qty`.

Q4. Is your database access actually parallel? No. There is one connection guarded by `dbMutex`, so DB work is serialized. Networking and HTML building are parallel. A connection pool would make DB work parallel.

Q5. How does the server know who you are? A random 128-bit session ID is stored in `user_sessions` and in an `HttpOnly` cookie. Every request looks it up with `is_active = TRUE`.

Q6. Why store sessions in MySQL and not in memory? They survive restarts, keep a login history, support multiple devices, and are easy to audit.

Q7. What happens if checkout fails halfway? `ROLLBACK` undoes every change made since `START TRANSACTION`. That is atomicity.

Q8. Why save `price` in `order_items` when `products` already has it? It is a historical snapshot. A later price change must not alter past orders.

Q9. What does `thread_local` do here? It gives each worker its own copy of the current session and user, so concurrent requests never mix.

Q10. How do you prevent SQL injection and XSS? `esc()` (`mysql_real_escape_string`) on SQL inputs, and `htmlEscape()` on HTML output. Prepared statements would be even stronger.

Q11. Can a deadlock occur? Not between our locks. Each lock is held briefly, and they are never acquired in a nested or circular order.

Q12. What does `UNIQUE(cart_id)` on `orders` achieve? It makes `cart` → `order` a 1-to-0..1 relationship. One cart can never produce two orders.

23. Glossary

Term	Simple meaning
Socket	An endpoint for sending and receiving data over a network
TCP	A reliable, ordered byte-stream network protocol; HTTP runs on it
HTTP	The text protocol browsers and servers use to talk
Port 8080	The "door number" our server listens on
Thread	An independent path of execution inside one program
Thread pool	A fixed set of reusable worker threads
Mutex	A lock; only one thread can hold it at a time
Semaphore	A counter that threads can wait on (sleep) until it is above zero

Term	Simple meaning
Critical section	Code that must not be run by two threads at the same time
Race condition	A bug where the result depends on thread timing
Deadlock	Threads waiting on each other forever
Session	Server-side record that says "this ticket belongs to user X"
Cookie	Small value the browser stores and sends back automatically
HttpOnly	Cookie flag: JavaScript cannot read the cookie
SameSite	Cookie flag: limits sending the cookie on cross-site requests
Transaction	A group of SQL statements that succeed or fail together
COMMIT / ROLLBACK	Make the transaction permanent / undo it
ACID	Atomicity, Consistency, Isolation, Durability
FOR UPDATE	Lock the selected rows until the transaction ends
Foreign key	A column that must match a primary key in another table
Normalization	Organising tables to avoid duplicate data
SQL injection	An attack that sneaks SQL code in through user input
XSS	An attack that sneaks JavaScript into a page through user input
CSRF	An attack that tricks a logged-in browser into sending an unwanted request